IC526 Advanced Compilers
Lec 16: Static Single Assignment

Yoonseung Kim
DGIST EECS
Fall 2026

Outline

- Why SSA?
- Dominance
- ϕ Nodes

Why SSA?

Same Text, Same Value?

RustY

```
fn f(a: i32, b: i32) -> i32 {
  let mut x = a + 1;
  let y = x * 2;
  let w = x * 2;
  x = b - 3;
  let z = x * 2;
  return y + w + z;
}
```

Can we reuse y?

```
fn f(a: i32, b: i32) -> i32 {
  let mut x = a + 1;
  let y = x * 2;
  let w = x * 2; // yes
  x = b - 3;
  let z = x * 2; // no
  return y + w y + z;
}
```

Q: How does the compiler tell w from z?

A: Dataflow Analysis, which becomes difficult with complex control flow

SSA: One Name, One Definition

Renamed

```
x1 = a + 1
y  = x1 * 2
w  = x1 * 2
x2 = b - 3
z  = x2 * 2
```

LLVM IR

```
%x1 = add i32 %a, 1
%y  = mul i32 %x1, 2
%w  = mul i32 %x1, 2
%x2 = sub i32 %b, 3
%z  = mul i32 %x2, 2
%s  = add i32 %y, %w
%r  = add i32 %s, %z
```

LLVM IR is in ***static single assignment (SSA)*** form:
every variable is assigned **exactly once** in the program text.

Reusing %y in SSA

Before

```
%x1 = add i32 %a, 1
%y  = mul i32 %x1, 2
%w  = mul i32 %x1, 2
%x2 = sub i32 %b, 3
%z  = mul i32 %x2, 2
%s  = add i32 %y, %w
%r  = add i32 %s, %z
```

After

```
%x1 = add i32 %a, 1
%y  = mul i32 %x1, 2

%x2 = sub i32 %b, 3
%z  = mul i32 %x2, 2
%s  = add i32 %y, %y
%r  = add i32 %s, %z
```

1. %y is computed first with the same operation and operands, so %w equals %y.

2. Replace every use of %w with %y.

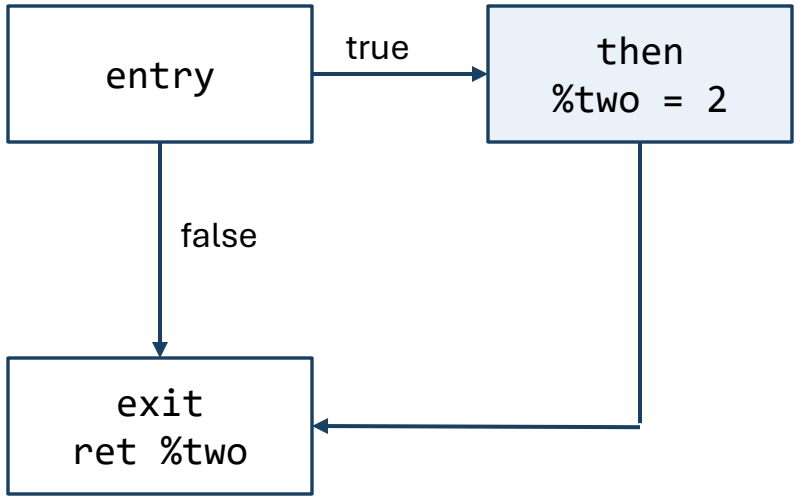
3. %w now has no users → delete it.

No dataflow analysis: comparing names is enough.

A Definition Available on Only One Path

Can we return a value defined in `then`?

```
entry:
  br i1 %c, label %then, label %exit
then:
  %two = add i32 1, 1
  br label %exit
exit:
  ret i32 %two
```

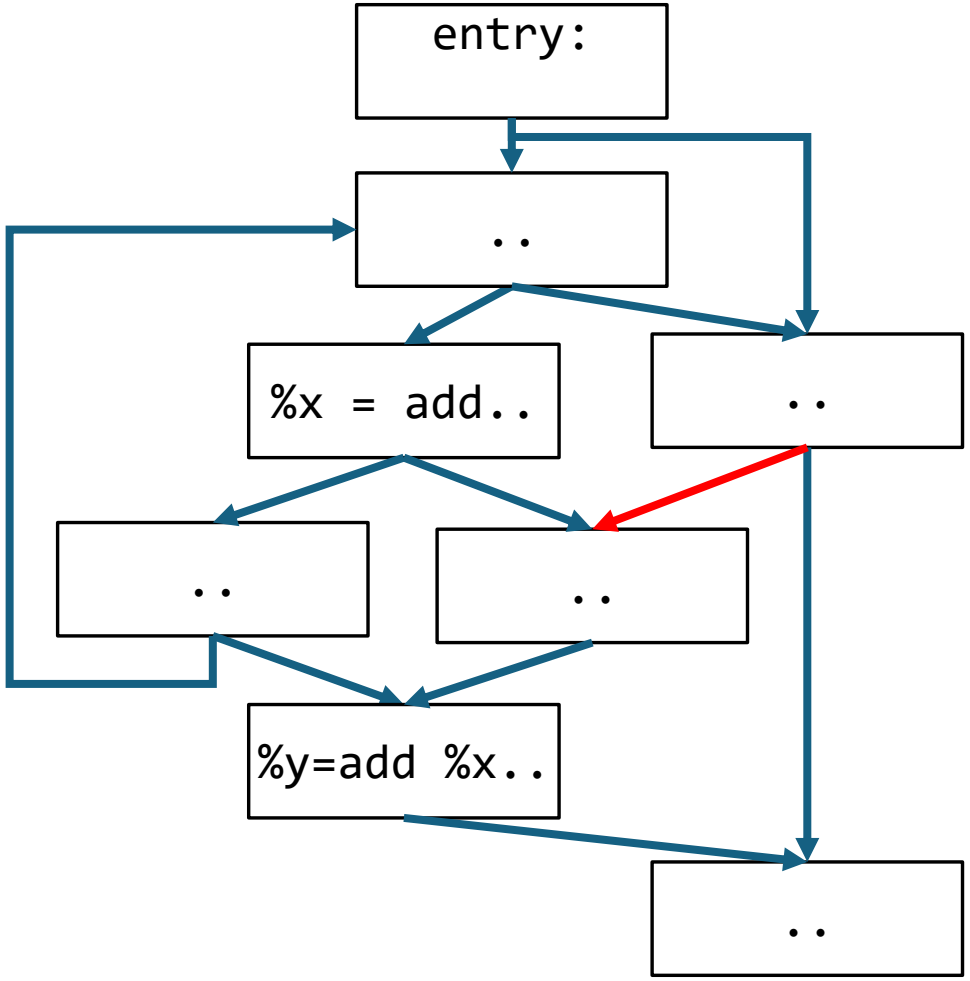


LLVM IR: a use must be ***dominated*** by its definition.

Dominance

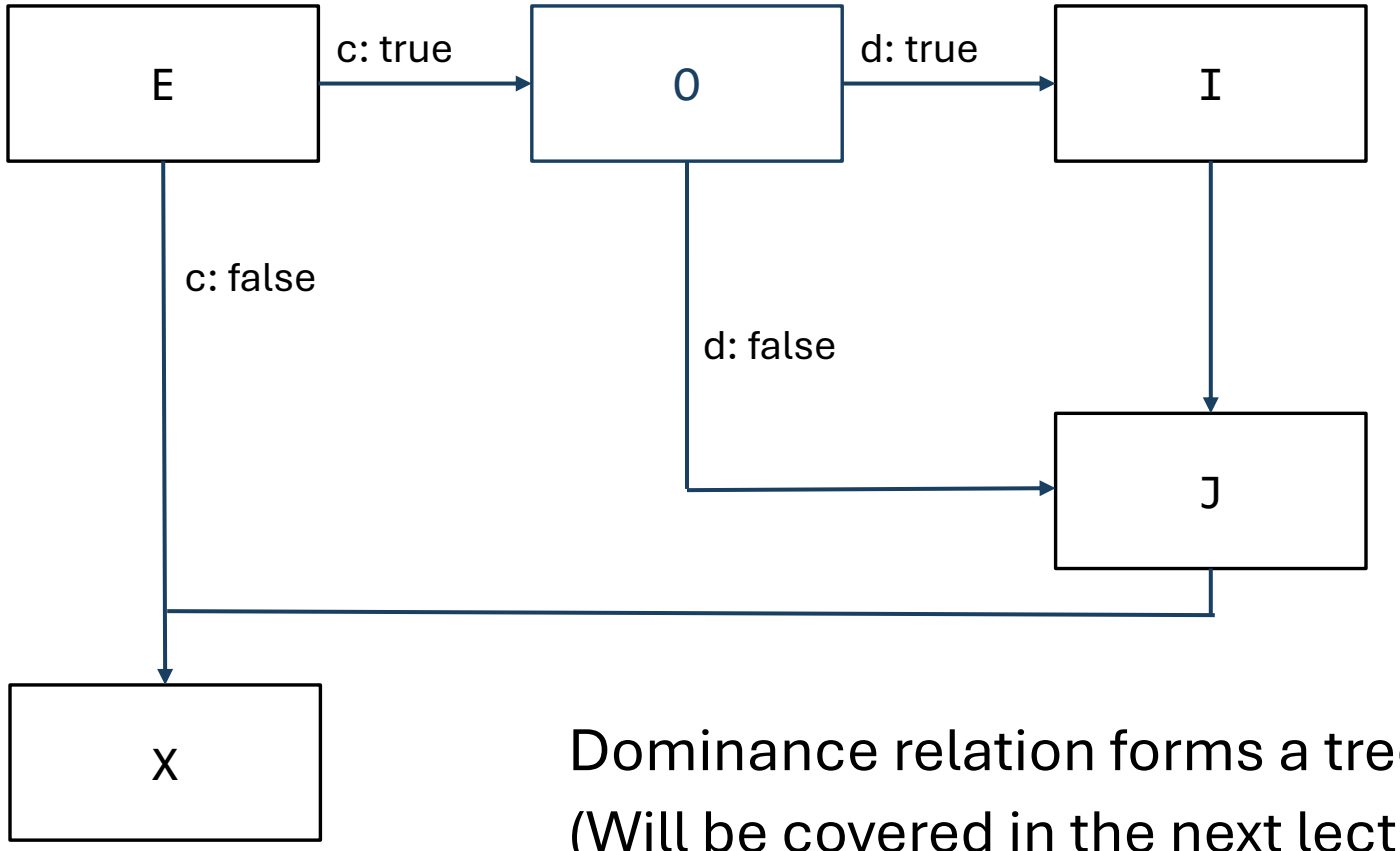
Dominance

A dominates B if every entry-to-B path passes through A.



Transitivity of Dominance Relation

A dominates B => every block that dominates A also dominates B



B	Dom(B)
E	{E}
O	{E, O}
I	{E, O, I}
J	{E, O, J}
X	{E, X}

Dominance relation forms a tree of blocks.
(Will be covered in the next lecture)

LLVM IR Enforces Dominance

The verifier rejects both: `Instruction does not dominate all uses!`

Across blocks

```
entry:
  br i1 %c, label %then, label %exit
then:
  %two = add i32 1, 1
  br label %exit
exit:
  ret i32 %two
```

Within a block

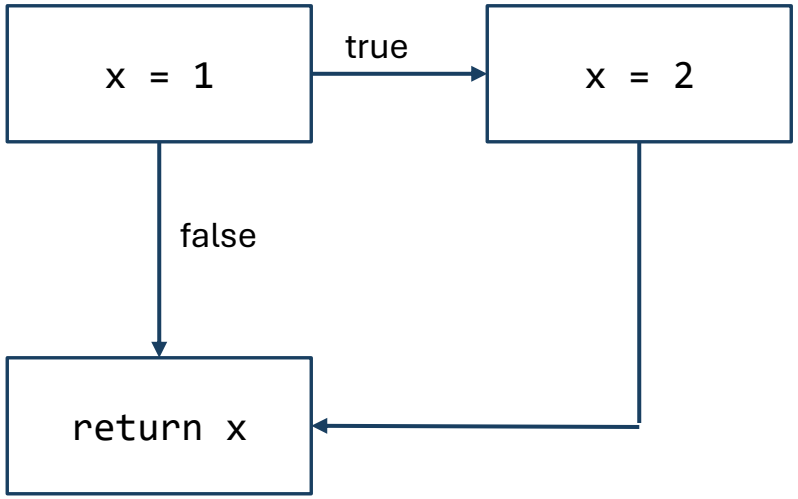
```
bb:
  %b = add i32 %a, 1   ; use
  %a = add i32 %x, 1   ; def
```

ϕ Nodes

Example: The Midterm Function

```
fn choose(c: bool) -> i32 {
  let mut x: i32 = 1;
  if c {
    x = 2;
  }
  return x;
}
```

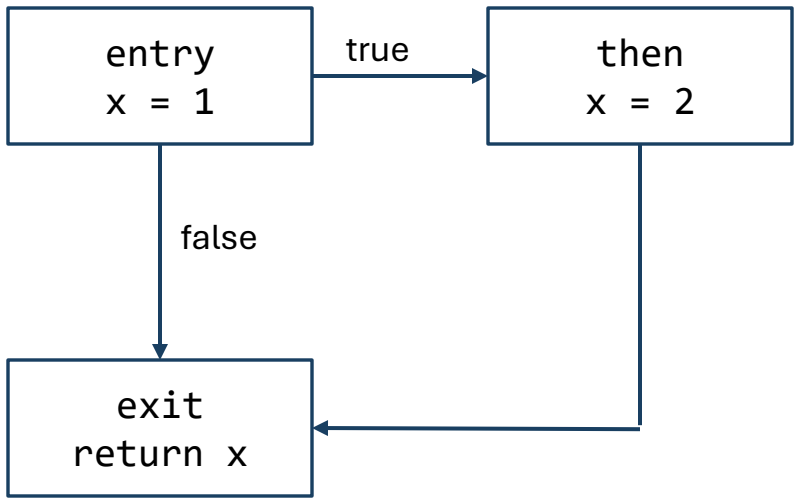
c	Return value
false	1
true	2



Q: Can we translate this into LLVM IR without memory accesses?

First Try: Naïve Translation

```
entry:
  %x = add i32 1, 0 ; no direct assignment
  br i1 %c, label %then, label %exit
then:
  %x = add i32 2, 0
  br label %exit
exit:
  ret i32 %x
```

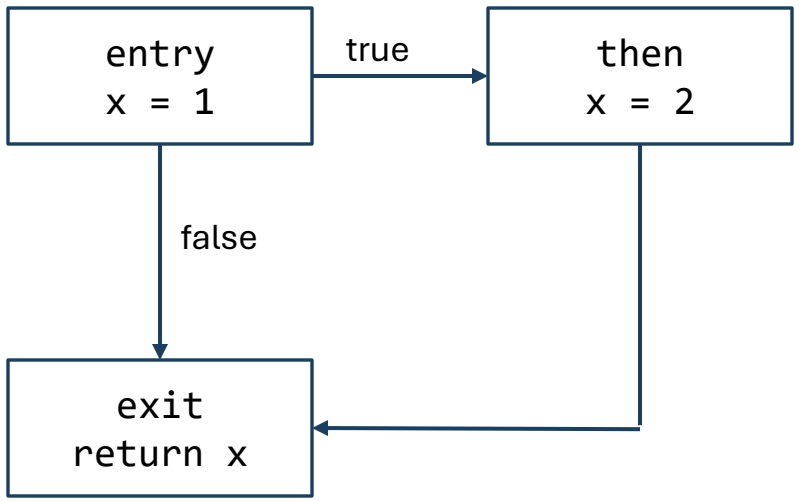


Prohibited by the SSA property

```
error: multiple definition of local value named 'x'
```

Second Try: Renaming

```
entry:
  %x = add i32 1, 0 ; no direct assignment
  br i1 %c, label %then, label %exit
then:
  %y = add i32 2, 0
  br label %exit
exit:
  %r = select i1 %c, i32 %y, i32 %x
  ret i32 %r
```

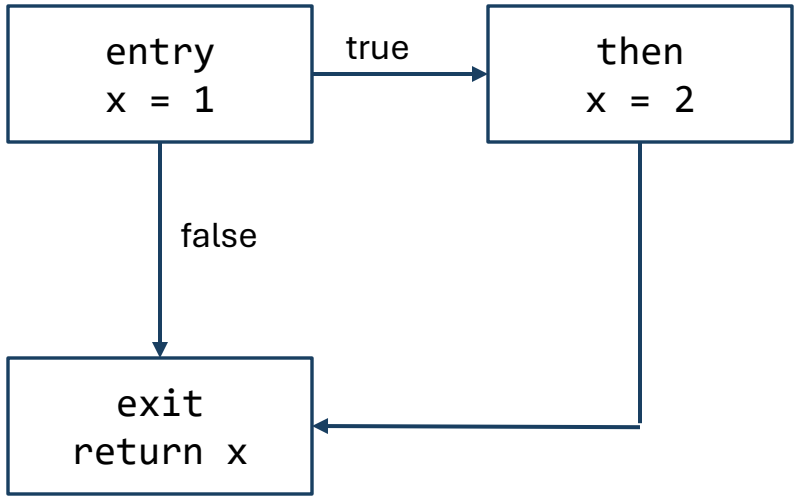


Prohibited by the **dominance rule**

Instruction does not dominate all uses!

Third Try: A New Instruction, ϕ (Phi Nodes)

```
entry:
  %x = add i32 1, 0
  br i1 %c, label %then, label %exit
then:
  %y = add i32 2, 0
  br label %exit
exit:
  %r = phi i32 [%x, %entry], [%y, %then]
  ret i32 %r
```



Accepted: ϕ picks the value of the incoming edge.

The ϕ Instruction

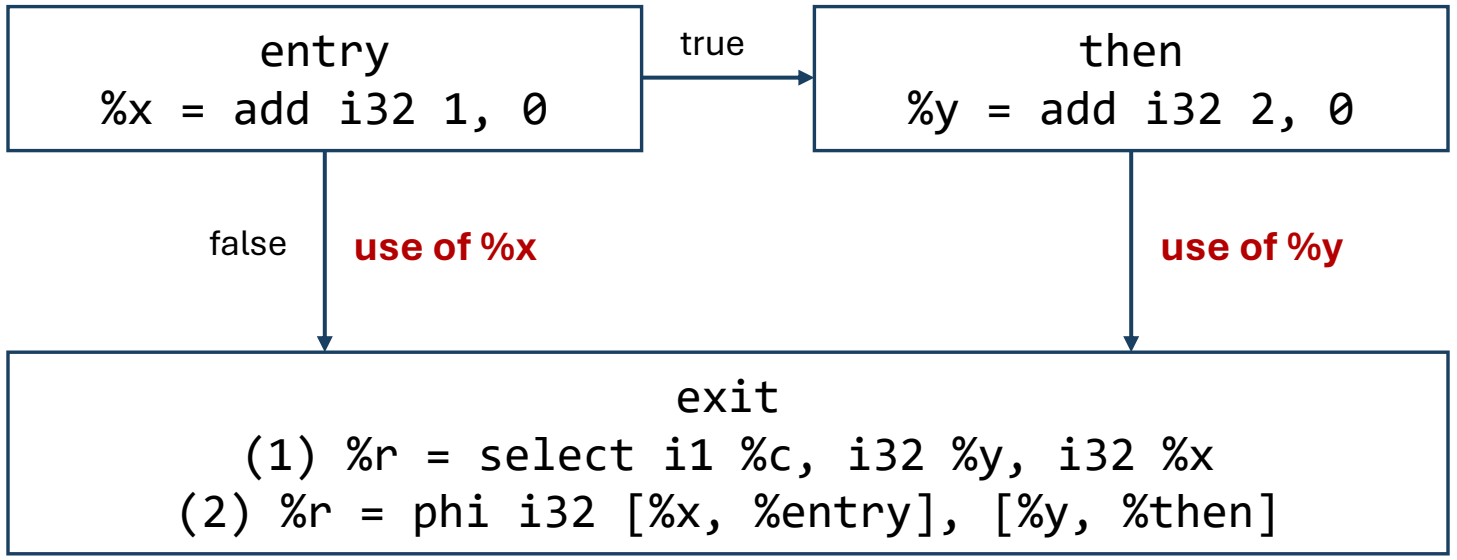
ϕ selects the value associated with the incoming edge.

ϕs come first in a block, with one entry per predecessor.

```
entry:
  br i1 %c, label %then, label %exit
then:
  %y = add i32 2, 0
  br label %exit
exit:
  %r = phi i32 [1, %entry], [%y, %then]
  ret i32 %r
```

Edge into exit	%r
entry → exit	1
then → exit	%y=2

Considering Dominance: Where Is a ϕ Operand Used?

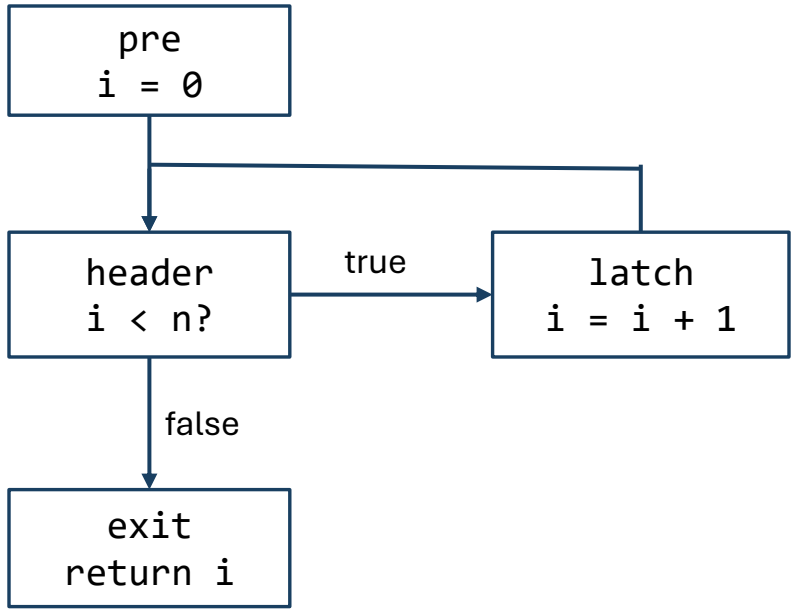


Φ operand counts as a use at the **end of predecessor**

	Use of %y is in	Requirement	OK?
(1) select	exit	then dom exit	✗
(2) ϕ	end of then	then dom then	✓

Example: A Loop-Carried Variable

```
fn count(n: i32) -> i32 {
  let mut i: i32 = 0;
  while i < n {
    i = i + 1;
  }
  return i;
}
```



The first i comes from `pre`;
every later I comes from `latch`.

A ϕ at the Loop Header

```
pre:
  br label %header
header:
  %i = phi i32 [0, %pre], [%next, %latch]
  %c = icmp slt i32 %i, %n
  br i1 %c, label %latch, label %exit
latch:
  %next = add i32 %i, 1
  br label %header
exit:
  ret i32 %i
```

n = 3

Visit	%i	From
1	0	pre
2	1	latch
3	2	latch
4	3	latch

Note: SSA enforces one **static** definition; each visit to the header makes a new %i.

Alive2: Checking ϕ Operands

Source (frontend output)

```
define i32 @src(i1 %c) {
E:
  %p = alloca i32
  store i32 1, ptr %p
  br i1 %c,label %T,label %X
T:
  store i32 2, ptr %p
  br label %X
X:
  %r = load i32, ptr %p
  ret i32 %r
}
```

Target (SSA with ϕ)

```
define i32 @tgt(i1 %c) {
E:
  br i1 %c,label %T,label %X
T:
  br label %X
X:
  %r = phi i32 [1,%E],[2,%T]
  ret i32 %r
}
```

Exercise 1: Construct the ϕ

RustY

```
fn choose(c: bool) -> i32 {
  let mut x: i32 = 5;
  if c {
    x = x + 3;
  }
  return x;
}
```

SSA LLVM IR

```
entry:
  br i1 %c, label %then, label %exit
then:
  %t = add i32 5, 3
  br label %exit
exit:
  %r = phi i32
    [?, %entry], [?, %then]
  ret i32 %r
```

Fill in the incoming values of the ϕ.

Exercise 1 Answer

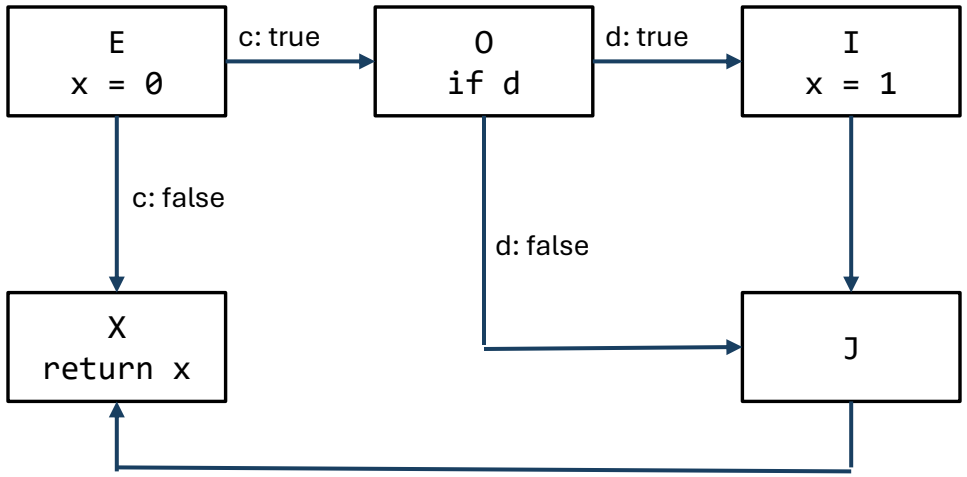
```
then:
  %t = add i32 5, 3
  br label %exit
exit:
  %r = phi i32 [5, %entry], [%t, %then]
  ret i32 %r
```

Each path's last definition of x becomes a ϕ input.

Constant folding could simplify %t to 8 in a separate step.

Exercise 2: Nested Conditionals

```
fn nested(c: bool,
          d: bool) -> i32 {
  let mut x: i32 = 0;
  if c {
    if d { x = 1; }
  }
  return x;
}
```



Where do ϕs go, and what are their incoming values?

Exercise 2 Answer

```
E:
  br i1 %c, label %O, label %X
O:
  br i1 %d, label %I, label %J
I:
  br label %J
J:
  %x.j = phi i32 [0,%O], [1,%I]
  br label %X
X:
  %x.e = phi i32 [0,%E], [%x.j,%J]
  ret i32 %x.e
```

c	d	Return
false	—	0
true	false	0
true	true	1

A ϕ input can be another ϕ: %x.j flows into %x.e.

Next Lecture: mem2reg

Source (frontend output)

```
define i32 @src(i1 %c) {
E:
  %p = alloca i32
  store i32 1, ptr %p
  br i1 %c,label %T,label %X
T:
  store i32 2, ptr %p
  br label %X
X:
  %r = load i32, ptr %p
  ret i32 %r
}
```

Target (SSA with ϕ)

```
define i32 @tgt(i1 %c) {
E:
  br i1 %c,label %T,label %X
T:
  br label %X
X:
  %r = phi i32 [1,%E],[2,%T]
  ret i32 %r
}
```

The mem2reg pass does this automatically.